

DML

INSERT, UPDATE, DELETE

학습목표

1. DML 문을 학습한다.
2. 날짜 입력에 대해 학습한다.
3. 행에 대한 조건별 수정 삭제에 대해서 학습한다.
4. 트랜잭션에 대해서 학습한다.

DML와 TCL

```
INSERT INTO <테이블> [(컬럼, 컬럼, ... )]  
VALUES (값, 값, ...);
```

```
UPDATE 테이블  
SET 컬럼 = 값, 컬럼 = 값, ...  
[WHERE 조건];
```

```
DELETE FROM 테이블  
[WHERE 조건];
```

```
COMMIT;
```

```
ROLLBACK;
```

DELETE와 TCL

- 예제 테이블에 데이터를 삭제하고 TCL을 통해 결과를 확인한다.

```
SQL> SELECT * FROM dept;
```

```
SQL> DELETE FROM dept;
```

```
SQL> SELECT * FROM dept;
```

```
SQL> ROLLBACK;
```

```
SQL> SELECT * FROM dept;
```

- DML 문은 반드시!! COMMIT이나 ROLLBACK을 사용해서 작업을 종료해야 한다.

```
SQL> DELETE FROM emp;
```

```
SQL> DELETE FROM dept;
```

```
SQL> COMMIT;
```


INSERT

- EMP 테이블에 아래 제공된 값을 입력한다.

ENO	ENAME	SEX	JOB	MGR	HDATE	SAL	COMM	DNO
1001	문시현	남	모델링		1991/02/01	4500	520	10
1002	김주란	여	모델링		1992/03/03	4100	330	20
1003	양선호	남	모델링		1995/02/21	4300		30

```
SQL> ALTER SESSION SET nls_date_format='YYYY/MM/DD';
SQL> DESC emp;
SQL> INSERT INTO emp (eno, ename, sex, job, mgr, hdate, sal, comm, dno)
  2  VALUES ('1001', '문시현', '남', '모델링', NULL, '1991/02/01', 4500, 520, '10');
SQL> INSERT INTO emp (eno, ename, sex, job, hdate, sal, comm, dno)
  2  VALUES ('1002', '김주란', '여', '모델링', '1992/03/03', 4100, 330, '20');
SQL> INSERT INTO emp
  2  VALUES ('1003', '양선호', '남', '모델링', NULL, '1995/02/21', 4300, NULL, '30');
SQL> SELECT * FROM emp;
SQL> COMMIT;
```

INSERT와 날짜 입력

```
SQL> ALTER SESSION SET nls_date_format='YYYY/MM/DD:HH24:MI:SS';
```

```
SQL> INSERT INTO emp (eno, ename, hdate)
  2  VALUES ('0001', '안영희', '2021/09/25:03:07:15');
```

```
SQL> INSERT INTO emp (eno, ename, hdate)
  2  VALUES ('0202', '손하늘', '2021/09/25');
```

```
SQL> ALTER SESSION SET nls_date_format='YYYY/MM/DD';
```

```
SQL> INSERT INTO emp (eno, ename, hdate)
  2  VALUES ('0201', '안영숙', '2021/09/25:03:07:15');
```

```
SQL> INSERT INTO emp (eno, ename, hdate)
  2  VALUES ('0201', '안영숙', TO_DATE('2021/09/25:03:07:15',
                                         'YYYY/MM/DD:HH24:MI:SS'));
```

- 날짜 입력에는 터미널의 날짜 형식에 따라 입력하는 것이 아니라 반드시 TO_DATE 함수를 이용해서 날짜를 입력한다.

INSERT와 날짜 입력

- 날짜 데이터에 적용되는 연도와 나머지 DEFAULT 값을 확인한다.

```
SQL> ALTER SESSION SET nls_date_format='YYYY-MM-DD:HH24:MI:SS';
```

```
SQL> INSERT INTO emp (eno,hdate)
  2  VALUES ('01', TO_DATE('2000', 'YYYY'));
```

```
SQL> INSERT INTO emp (eno,hdate)
  2  VALUES ('02', TO_DATE('99', 'YY'));
```

```
SQL> INSERT INTO emp (eno,hdate)
  2  VALUES ('03', TO_DATE('99', 'RR'));
```

```
SQL> INSERT INTO emp (eno,hdate)
  2  VALUES ('04', sysdate);           ← 05, TRUNC(sysdate)
```

```
SQL> SELECT * FROM emp
  2  WHERE eno IN ('01','02','03','04','05');
```

UPDATE, DELETE

- 김주란의 부서 번호를 10번으로, 급여는 10% 인상한다.
 - 실습을 위해 예제 테이블을 초기화한다.

```
SQL> UPDATE emp SET dno = '10', sal = sal*1.1  
2  WHERE ename = '김주란';  
  
SQL> COMMIT;
```

- 10번 부서 사원의 정보를 삭제한다.

```
SQL> DELETE FROM emp  
2  WHERE dno = '10';  
  
SQL> COMMIT;
```

실습

1. 다음 INSERT 문을 실행하고 입력 결과를 확인한다.

```
SQL> INSERT INTO emp (eno,hdate) VALUES (5, TO_DATE('12', 'MM'));
SQL> INSERT INTO emp (eno,hdate) VALUES (6, TO_DATE('10', 'DD'));
SQL> INSERT INTO emp (eno,hdate) VALUES (7, TO_DATE('9', 'HH24'));
SQL> INSERT INTO emp (eno,hdate) VALUES (8, TO_DATE('12', 'MI'));
SQL> INSERT INTO emp (eno,hdate) VALUES (9, TO_DATE('20', 'SS'));
SQL> INSERT INTO emp (eno,hdate) VALUES (10, sysdate);
```

2. Emp 테이블의 행을 모두 삭제하고 다음 제시된 데이터를 입력한다.

2001	남궁연호	남	개발	0202	1993/12/13	3950	200	10
2002	제갈민	남	개발	0202	1996/04/30	1520	2000	20
2003	정의찬	남	개발	0202	1992/03/03	4350		30
2007	이초록	남	개발	0001	1992/09/05	1989	2300	30
2008	윤고은	여	개발	0001	1992/03/03	2100		40

3. 모든 학생의 성적을 4.5만점 기준으로 수정한다.
4. 모든 교수의 부임일자를 100일 앞으로 수정한다.
5. 화학과 2학년 학생의 정보를 삭제한다.
6. 조교수의 정보를 삭제한다.

Transaction과 Lock

학습목표

1. 트랜잭션의 개념에 대해서 학습한다.
2. 트랜잭션의 특성에 대해서 학습한다.
3. 언두(UNDO) 정보와 UNDO SEGMENT에 대해서 학습한다.
4. 트랜잭션으로 인한 대기 현상에 대해서 실습한다.

트랜잭션(Transaction)

- 정의

- 반드시 함께 실행 되어야 할 작업의 단위

- 특성

- 원자성(Atomicity)

- 트랜잭션은 최소의 작업 단위로서 전체가 처리되거나 취소될 수 있지만 일부만 처리될 수 없다.

- 일관성(Consistency)

- 트랜잭션이 실행된 이후 데이터베이스의 무결성은 반드시 유지되어야 한다.

- 독립성(Isolation)

- 트랜잭션을 여러 개 동시에 실행하더라도 각각의 트랜잭션은 서로 영향을 줄 수 없다.
- 즉 실행이 종료되지 않은 트랜잭션의 결과는 다른 트랜잭션에서 참조하는 것이 불가능하다.

- 영속성(Durability)

- 종료된 트랜잭션의 결과는 반드시 데이터베이스에 반영되어야 한다.

트랜잭션의 시작과 종료

- 시작

- 이전 트랜잭션이 종료된 이후 DML(INSERT, UPDATE, DELETE)문장이나 DDL(CREATE, ALTER, DROP, TRUNCATE), DCL(GRANT, REVOKE)문장에 실행됐을 때 시작된다.

- 종료

- COMMIT이나 ROLLBACK 명령이 실행 될 때 종료된다.
- DDL이나 DCL문장의 실행이 완료되면 자동으로 종료된다.
- 사용자의 정상 종료 시에 종료된다.
- 데드락(Deadlock)이 걸리면 트랜잭션의 일부만 종료된다.

트랜잭션은 하나의 세션에서 단지 하나만 시작할 수 있으며 DML인 경우 반드시 COMMIT이나 ROLLBACK으로 종료 하지만 DDL이나 DCL의 경우 문장이 실행되고 난 후 자동으로 종료된다. 오라클은 SAVE POINT명령을 이용 트랜잭션의 과정을 제어할 수 있다.

트랜잭션과 언두 세그먼트(Undo segment)

트랜잭션과 DML 데이터

- INSERT등의 DML 작업이 실행되면 즉시 테이블에 적용된다
- 변경된 행에는 독점 잠금(Exclusive lock)이 발생한다.
- 해당 테이블에는 공유 잠금 (Share lock)이 발생한다.
- 트랜잭션이 종료되면 잠금이 해제된다.

UNDO SEGMENT

- 트랜잭션이 실행 될 때 이전 이미지를 저장한다.
- Rollback을 위해 트랜잭션이 수행되는 동안 이전 정보를 저장한다.

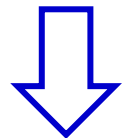
세그먼트

- 물리적인 저장 영역을 가진 오브젝트를 의미한다.
- 테이블, 인덱스 등 저장 영역에 공간을 할당 받는 오브젝트를 의미한다.

트랜잭션과 언두 세그먼트(Undo segment)

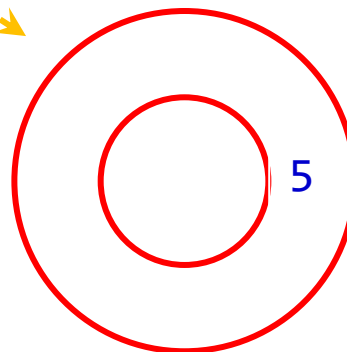
UPDATE 테이블
SET 컬럼 = 7
WHERE 조건

		5		



테이블과 행에
잠금 발생

		7		



- ① undo segment 결정
- ② 5 undo seg에 저장
- ③ 7 테이블에 저장
- ④ 행에 독점 잠금
- ⑤ 테이블에 공유 잠금

-
- ⑥ COMMIT : 7 영구저장
 - 잠금 해제
 - ⑦ ROLLBACK : 5 복원
 - 잠금 해제

트랜잭션과 잠금의 이해

- 실습은 두 개의 터미널을 이용한다.
 - 동일한 계정으로 접속한다.

```
SQL> SELECT eno, ename, sal FROM emp WHERE ename = '문시현';
```

```
SQL> UPDATE emp SET sal = sal*2 WHERE ename = '문시현';
```

```
SQL> SELECT eno, ename, sal FROM emp WHERE ename = '문시현';
```

```
SQL> SELECT eno, ename, sal FROM emp WHERE ename = '문시현';
```

```
SQL> COMMIT;
```

```
SQL> SELECT eno, ename, sal FROM emp WHERE ename = '문시현';
```

잠금으로 인한 대기 현상

```
SQL> SELECT eno, ename, sal FROM emp WHERE ename = '문시현';
```

```
SQL> UPDATE emp SET sal = sal*2 WHERE ename = '문시현';
```

```
SQL> SELECT eno, ename, sal FROM emp WHERE ename = '문시현';
```

```
SQL> UPDATE emp SET COMM = 1000 WHERE ename = '문시현';
```

← 대기 상태에 빠짐

```
SQL> COMMIT;
```

데드락

```
SQL> SELECT * FROM emp WHERE ename IN ('안영희', '문시현');  
SQL> UPDATE emp SET sal = sal*1.2 WHERE ename = '문시현';
```

```
SQL> SELECT * FROM emp WHERE ename IN ('안영희', '문시현');  
SQL> UPDATE emp SET sal = sal*2 WHERE ename = '안영희';
```

```
SQL> UPDATE emp SET COMM = 1000 WHERE ename = '안영희';  
← 대기 상태
```

```
SQL> UPDATE emp SET JOB = '경영' WHERE ename = '문시현';  
← 데드락 발생
```

- 데드락이 발생된 원인을 강제로 ROLLBACK 시킨다.

테이블 생성과 데이터 타입

학습목표

1. 테이블 생성, 삭제 명령을 학습한다.
2. 컬럼의 데이터 타입을 학습한다.
3. 테이블 생성시 주의 사항을 학습한다.
4. DROP된 테이블을 복구하는 방법을 학습한다.
5. 이후 과정에 사용될 실습용 스크립트를 생성한다.

DDL

- DDL(Data Definition Language)
 - CREATE
 - ALTER
 - DROP
 - RENAME
 - TRUNCATE

테이블 생성과 삭제

```
CREATE TABLE 테이블 (  
  컬럼 데이터_타입 [DEFAULT 값] [컬럼 레벨 제약조건],  
  .....  
  [테이블 레벨 제약조건],  
  .....  
);
```

```
DROP TABLE 테이블  
[CASCADE CONSTRAINT];
```

```
PURGE RECYCLEBIN;
```

테이블 생성 규칙

- 문자로 시작한다.
- 30자 이내로 한다.
- 영문, 숫자, _, \$, # 을 사용한다.
 - 한글 사용은 가능하지만 되도록 사용하지 않는 것이 좋다.
- 테이블의 이름은 동일한 유저(스키마) 안에서 유일해야 한다.
- 예약어는 사용이 불가능하다.
- 대소문자를 구별하지 않는다.
 - 생성할 때 사용한 문자와는 관계없이 모든 이름은 대문자로 정의된다.
 - 테이블 이름은 디렉너리에 저장되는데, 모두 대문자로 저장된다.

데이터 타입

VARCHAR2(n)	가변 길이 문자 타입 (1 < n < 4000 byte)
CHAR(n)	고정 길이 문자 타입 (1 < n < 2000 byte)
NUMBER(n,p)	숫자 타입, n은 전체 자리수이고 p는 소수점 이하 자리수이다. 전체 자리수를 초과할 경우 입력 거부되지만 소수점 이하 자리수가 초과되면 반올림되어 입력된다.
DATE	날짜 타입, 출력이나 입력 형식과 무관하게 YYYY/MM/DD:HH24:MI:SS 형태로 저장된다. (기원전 4712년 1월1일 ~ 서기 9999년 12월 31일)
LONG	2GB까지 저장 가능한 가변길이 문자 타입, 조건 검색할 수 없으며 테이블에는 하나의 LONG 컬럼만 정의할 수 있다.
CLOB	LONG를 개선한 타입, 최대 4G까지 저장 가능하고 한 테이블의 여러 컬럼에 정의할 수 있다.
BLOB	4G 까지 저장 가능한 가변길이 이진 타입
ROW(n)	가변길이 이진 타입
BFILE	4G 이내 의 외부 파일 저장을 위한 이진 타입
ROWID	ROWID를 저장하기 위한 데이터 타입, 각 문자는 64진수로 인코딩 되어 있다.

테이블 생성 관련 명령어

```
SQL> SELECT * FROM tab;
```

```
SQL> SELECT table_name FROM user_tables;
```

- 스키마에 테이블 목록을 검색한다.
- 스키마는 유저와 동일하게 쓰인다.

```
SQL> DESC 테이블;
```

```
SQL> SELECT table_name, column_name, data_type, data_length  
2 FROM user_tab_columns  
3 [WHERE table_name = '테이블'];
```

- 테이블의 컬럼 구성을 확인한다.

테이블 생성과 확인

- 다음과 같은 구조의 테이블을 생성하고 확인한다.

테이블명 : board(게시판)

구성 컬럼 : no(게시물번호), name(작성자), sub(제목), content(내용), hdate(입력일시)

```
SQL> CREATE TABLE board (  
2   no NUMBER,  
3   name VARCHAR2(50),  
4   sub VARCHAR2(100),  
5   content VARCHAR2(4000),  
6   hdate DATE DEFAULT SYSDATE  
7 );
```

```
SQL> SELECT * FROM tab;
```

```
SQL> SELECT table_name FROM user_tables  
2  WHERE table_name = 'BOARD';
```

테이블 생성과 확인

- 생성된 테이블에 행을 입력하고 default 입력을 확인한다.

```
SQL> DESC board;
```

```
SQL> SELECT table_name, column_name, data_type, data_length  
2   FROM user_tab_columns  
3   WHERE table_name = 'BOARD';
```

```
SQL> INSERT INTO board (no) VALUES (1);
```

```
SQL> COMMIT;
```

```
SQL> SELECT * FROM board;
```

문자 타입 데이터

- 문자와 숫자
 - 주민번호, 군번, 숫자만으로 구성된 학번 : 문자
 - . 개별 자리가 의미가 있는 경우 항상 문자형 데이터
 - . 연산 가능성이 없다.
 - 금액 : 숫자
 - . 연산 가능성이 있으면 숫자
 - . NULL에 주의 한다.
- VARCHAR, VARCHAR2
 - 같은 데이터 타입
 - 앞으로 뭔가 변경될 것이다.???
- CHAR, VARCHAR2
 - CHAR의 장점은 없다.
 - VARCHAR2를 쓴다.

CHAR, VARCHAR2의 비교

- 동일한 값이 입력된 두 컬럼이 데이터 타입에 따라 다르게 인식됨을 확인한다.

```
SQL> CREATE TABLE comp(  
2   co1 CHAR(4),  
3   co2 VARCHAR2(4)  
4 );
```

A	A		
---	---	--	--

```
SQL> INSERT INTO comp VALUES ('AA', 'AA');
```

A	A
---	---

```
SQL> SELECT LENGTHB(co1), LENGTHB(co2) FROM comp;
```

```
SQL> SELECT * FROM comp WHERE co1='AA';
```

```
SQL> SELECT * FROM comp WHERE co2='AA';
```

```
SQL> SELECT * FROM comp WHERE co1=co2;
```

날짜 타입 컬럼 검색

- 날짜 데이터 입력시에 sysdate등의 자동으로 입력되는 기능을 이용할 때 주의 사항

```
SQL> CREATE TABLE hd (  
2   no NUMBER,  
3   hdate DATE  
4 );
```

```
SQL> INSERT INTO hd VALUES (1, sysdate);
```

```
SQL> SELECT * FROM hd;
```

```
SQL> SELECT * FROM hd WHERE hdate = '2021/10/02';
```

```
SQL> SELECT no, TO_CHAR(hdate, 'YYYY/MM/DD:HH24:MI:SS') FROM hd;
```

```
SQL> SELECT * FROM hd  
2  WHERE hdate BETWEEN '2021/10/02' AND '2021/10/03';
```

- 더 좋지 못한 조건문 : WHERE hdate like '2021/10/02%';

테이블 삭제

- board, comp, hd 테이블을 완전 삭제한다.

```
SQL> SELECT * FROM tab;
```

```
SQL> DROP TABLE board;
```

```
SQL> DROP TABLE comp;
```

```
SQL> DROP TABLE hd;
```

```
SQL> SELECT * FROM tab;
```

```
SQL> PURGE RECYCLEBIN;
```

```
SQL> SELECT * FROM tab;
```

삭제 테이블 복구

```
SELECT object_name, original_name FROM recyclebin;  
  
FLASHBACK TABLE ORIGINAL_NAME TO BEFORE DROP  
[RENAME TO NEW_NAME];
```

- 휴지통에 삭제된 테이블명을 확인하고 삭제된 테이블을 복구한다.
- TEST 테이블을 삭제하고 복구한다.

```
SQL> DROP TABLE test;  
SQL> SELECT * FROM tab;  
SQL> SELECT object_name, original_name FROM recyclebin;  
SQL> FLASHBACK TABLE test TO BEFORE DROP;  
SQL> SELECT * FROM tab;
```

테이블 생성 실습 방법

- 테이블 생성과 같이 긴 SQL문은 반드시 스크립트 파일을 만들어 두고 실행한다.
- 스크립트 파일 생성시 주의 사항
 - 파일명은 ###.sql 파일로 생성한다.
 - 반드시 스크립트 파일을 저장하는 별도의 폴더를 지정하거나 sqlplus를 실행하는 sql 폴더에 저장한다.

실습

1. 예제를 통해 만들어진 테이블을 확인하고 모두 삭제한다.
2. emp, dept, salgrade, student, professor, course, score 테이블을 생성한다.
 - 테이블 구조는 DESC 명령을 이용해 확인한다.
 - 테이블에는 제약조건 등을 생성하지 말고 컬럼만 구성한다.
 - 생선전에 미리 테이블을 삭제하고 명령은 반드시 스크립트를 통해서 생성한다.
 - 생성된 스크립트는 다음 실습에 사용된다.

테이블 관리

학습목표

1. 서브 쿼리를 이용해서 테이블을 만드는 과정을 학습한다.
2. 테이블의 컬럼을 추가, 수정, 삭제하는 과정을 학습한다.
3. TRUNCATE 명령과 DELETE TABLE 명령의 차이를 학습한다.
4. OBJECT의 이름을 바꾸는 RENAME 명령을 학습한다.

서브 쿼리를 이용한 테이블 생성

```
CREATE TABLE 테이블 [(  
  컬럼 [DEFAULT 값],  
  ...  
)]  
AS (SELECT 문장 : Sub query문);
```

- 서브 쿼리 실행 결과를 테이블로 생성한다.
- 컬럼 목록이 생략되면 서브쿼리의 열 이름이 생성될 테이블의 컬럼 명이 된다.
- 컬럼에 DEFAULT 항목을 지정할 수 있다.
- 테이블을 복사하거나 일부를 별도로 저장하는 용도로 사용한다.

서브 쿼리를 이용한 테이블 생성

- 사번, 이름, 업무, 부서 정보만을 갖는 테이블을 생성한다.

```
SQL> CREATE TABLE emp2
  2  AS
  3  SELECT eno 사번, ename 이름, job 업무, d.dno 부서번호, dname 부서
  4  FROM emp e, dept d
  5  WHERE d.dno=e.dno;
SQL> DESC emp2;

SQL> SELECT * FROM emp2;
```

```
SQL> CREATE TABLE emp2 (사번, 이름, 업무, 부서번호, 부서)
  2  AS
  3  SELECT eno, ename, job, d.dno, dname
  4  FROM emp e, dept d
  5  WHERE d.dno=e.dno;
```

테이블 관리(ALTER TABLE)

```
ALTER TABLE 테이블  
ADD (컬럼 데이터_타입 [DEFAULT 값], ...);
```

```
ALTER TABLE 테이블  
MODIFY (컬럼 데이터_타입 [DEFAULT 값], ...);
```

```
ALTER TABLE 테이블  
DROP COLUMN 컬럼;
```

- ALTER TABLE 명령
 - 컬럼 추가
 - 컬럼 수정
 - 컬럼 삭제

컬럼 추가

- emp2 테이블에 입사일 컬럼을 추가한다.

```
SQL> ALTER TABLE emp2  
2 ADD (hdate DATE);
```

```
SQL> DESC emp2;
```

```
SQL> SELECT * FROM emp2;
```

- 추가되는 열은 마지막에 위치한다.
- 열의 위치를 지정할 수 없다.
- 추가된 열에는 NULL이 저장된다.

컬럼 변경

- emp2. hdate 컬럼에 SYSDATE를 DEFAULT로 정의한다.

```
SQL> ALTER TABLE emp2  
2 MODIFY (hdate DEFAULT SYSDATE);
```

```
SQL> DESC emp2;
```

```
SQL> SELECT * FROM emp2;
```

- 기존 데이터 수용이 가능하면 데이터 타입, 길이 수정이 가능하다.
- 행이 없으면 자유로운 수정이 가능하다.
- 수정된 컬럼은 이후 행 입력부터 적용된다.
- 단 그런 작업을 할 필요가 없는 것이 정상이다.

컬럼 삭제

- emp2. 부서번호 컬럼을 삭제한다.

```
SQL> ALTER TABLE emp2  
2 DROP COLUMN 부서번호;
```

```
SQL> DESC emp2;
```

```
SQL> SELECT * FROM emp2;
```

- 데이터 유무와 상관없이 컬럼 삭제가 가능하다..
- 한번에 하나씩 삭제 가능하다.
- 테이블에 반드시 하나 이상의 컬럼이 존재해야한다.
- 삭제된 컬럼은 복구 할 수 없다.
- sys 계정의 테이블에서는 작업이 불가능하다.
- 단 이런 작업을 할 필요가 없는 것이 정상이다.

테이블 내용 완전 삭제

```
TRUNCATE TABLE 테이블;
```

- 테이블의 내용(행, 공간)을 완전히 삭제한다.
 - DDL 작업으로 ROLLBACK 할 수 없다.
 - 'DELETE FROM 테이블' 과 비슷하지만 공간까지 해제하는 차이가 있다.
 - . DELETE 에 비해 속도가 빠르다.
- emp2 테이블의 내용과 구조를 완전히 삭제한다.

```
SQL> TRUNCATE TABLE emp2;
```

테이블 이름 변경

```
RENAME OLD_NAME TO NEW_NAME;
```

- 테이블, 뷰(VIEW), 시퀀스등 다양한 대상의 이름을 변경 할 수 있다.
- emp2 테이블의 이름을 eemp로 수정한다.

```
SQL> RENAME emp2 TO eemp;  
SQL> SELECT * FROM tab;
```


제약 조건 1.

PK, FK

학습목표

1. PK, FK에 대해서 학습한다.
2. PK, FK를 통해 구현되는 무결성에 대해서 학습한다.
3. 테이블 구조와 ERD에 대해서 학습한다.
4. 테이블 구조와 테이블 상세 도표에 대해서 학습한다.

제약 조건의 이해

- 제약 조건
 - 테이블 단위에서 정의되고 적용된다.
 - 종속성이 존재하는 경우 테이블의 삭제를 막아준다.
 - 자료가 삽입, 갱신, 삭제될 때마다 규칙이 적용된다.
 - 일시적으로 활성화하거나 비활성화하는 것이 가능하다.
 - 제약 조건은 개체처럼 관리되므로 반드시 이름이 필요하다.
- 오라클에서 제공되는 제약 조건
 - PRIMARY KEY
 - FOREIGN KEY
 - UNIQUE KEY
 - NOT NULL
 - CHECK

PK, FK

- PK

- Primary Key, 주키, 주식별자
- 테이블마다 한 개만 정의 할 수 있다.
- 테이블내에 모든 행을 유일하도록 식별해주는 컬럼
- 모든 컬럼은 PK 컬럼에 함수적 종속 관계(functional dependency)를 갖는다.
 - . 학번 → (이름, 학년, 학과, ...)
- 중복 될 수 없고 NULL을 허용하지 않는다.
- 고유 인덱스가 자동으로 생성된다.

- FK

- Foreign Key, 외부키, 외부식별자
- 테이블 간 관계(Relationship)를 의미한다.
- 항상 부모 자식 관계이다.
- 자식 테이블의 참조 컬럼에 지정한다.
- 두 컬럼에 데이터 타입이 일치해야 한다.
- PK나 UK만 참조 가능하다.

PK, FK 설정

```
SQL> CREATE TABLE 테이블 (  
2     .....  
3     CONSTRAINT 제약_조건 PRIMARY KEY (컬럼));
```

```
SQL> CREATE TABLE 테이블 (  
2     .....  
3     CONSTRAINT 제약_조건 FOREIGN KEY (컬럼)  
4     REFERENCES 참조할_테이블 (참조할_컬럼) [ON DELETE CASCADE]);
```

```
SQL> CREATE TABLE 테이블 (  
2     컬럼 데이터_타입 CONSTRAINT 제약_조건 PRIMARY KEY,  
3     .....)
```

```
SQL> CREATE TABLE 테이블 (  
2     컬럼명 데이터_타입 CONSTRAINT 제약_조건 FOREIGN KEY  
3     REFERENCES 참조할_테이블 (참조할_컬럼)  
4     [ON DELETE CASCADE],  
5     .....)
```

제약 조건 검색

```
SQL> SELECT c.table_name, c.constraint_name, c.constraint_type,  
2         c.status, s.column_name  
3 FROM user_constraints c, user_cons_columns s  
4 WHERE c.constraint_name = s.constraint_name  
5 AND c.table_name in (검색_대상_테이블_목록)  
6 ORDER BY c.table_name;
```

```
SQL> SELECT  p.table_name 상위테이블, p.constraint_name 상위제약조건,  
2         c.table_name 하위테이블, c.constraint_name 참조제약조건  
3 FROM user_constraints p, user_constraints c  
4 WHERE c.r_constraint_name=p.constraint_name  
5 AND p.table_name in (검색_대상_테이블_목록)  
6 ORDER BY p.table_name;
```

- USER_CONSTRAINTS, USER_CONS_COLUMNS 로 부터 제약 조건을 검색한다.
- ucon.sql, ref.sql 스크립트로 저장해서 사용한다.

PK와 FK 설정 테이블 생성

- dept, emp 테이블을 생성한다.
 - 테이블 생성전에 두 테이블을 삭제하고 PK, FK를 추가해서 테이블을 생성한다.

```
SQL> DROP TABLE emp;  
SQL> DROP TABLE dept;  
SQL> PURGE RECYCLEBIN;
```

```
SQL> CREATE TABLE dept (  
2   dno VARCHAR2(4),  
3   dname VARCHAR2(30),  
4   loc VARCHAR2(18),  
5   CONSTRAINT dept_dno_pk PRIMARY KEY(dno)  
6 );
```

PK와 FK 설정 테이블 생성

```
SQL> CREATE TABLE emp (  
2   eno VARCHAR2(8),  
3   ename VARCHAR2(30),  
4   sex VARCHAR2(8),  
5   job VARCHAR2(24),  
6   mgr VARCHAR2(8),  
7   hdate DATE,  
8   sal NUMBER,  
9   comm NUMBER,  
10  dno VARCHAR2(4),  
11  CONSTRAINT emp_eno_pk PRIMARY KEY (eno),  
12  CONSTRAINT emp_mgr_fk FOREIGN KEY (mgr) REFERENCES emp (eno),  
13  CONSTRAINT emp_dno_fk FOREIGN KEY (dno) REFERENCES dept (dno)  
14 );
```

```
SQL> @ucon
```

```
SQL> @ref
```


제약 조건에 의한 무결성 통제

```
SQL> INSERT INTO dept (dno, dname, loc) VALUES ('10', '개발', '서울');  
SQL> INSERT INTO emp (eno, ename, dno) VALUES ('2000', '문시현', '10');  
SQL> COMMIT;
```

```
SQL> INSERT INTO dept (dno, dname, loc) VALUES ('10', '총무', '부산');  
← ORA-00001: 무결성 제약 조건.... 에러 발생
```

```
SQL> INSERT INTO emp (eno, ename, dno) VALUES ('2001', '손하늘', '20');  
← ORA-02291: 무결성 제약 조건.... 에러 발생
```

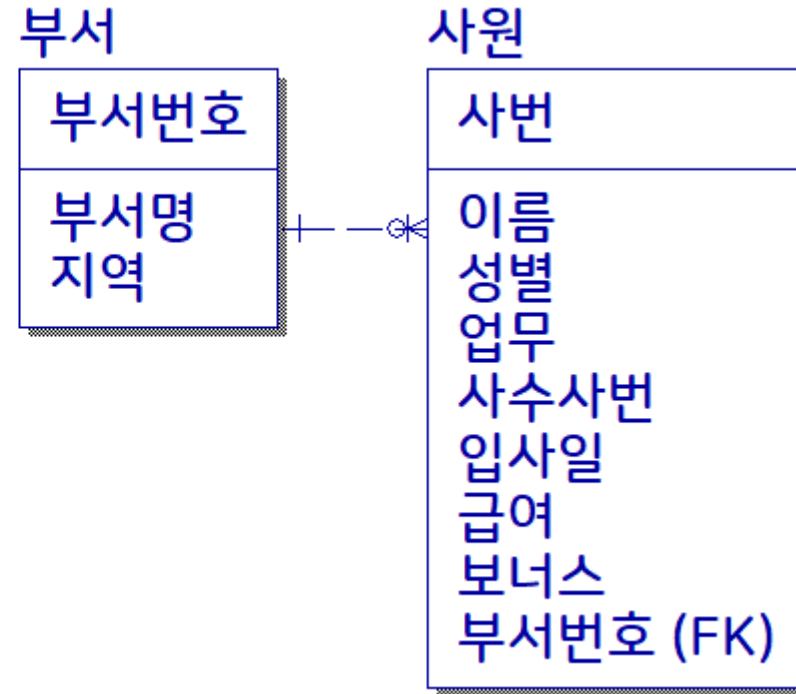
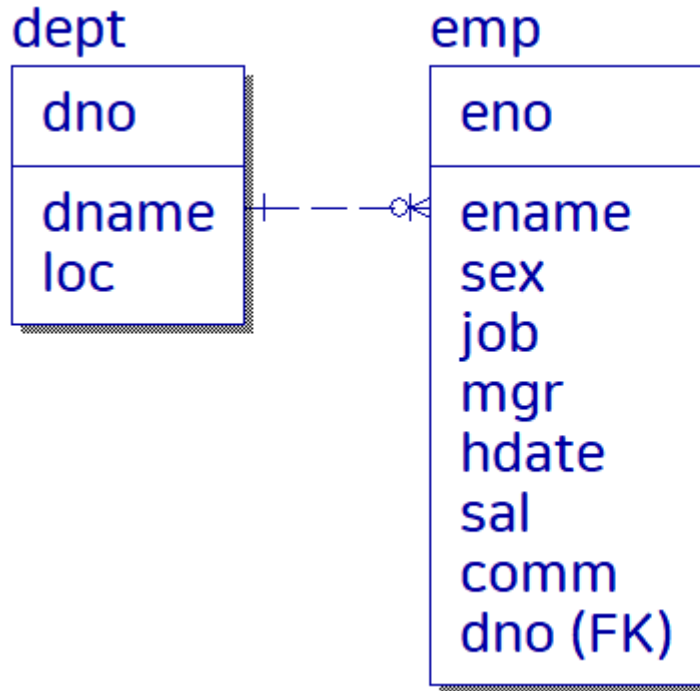
```
SQL> DELETE FROM dept WHERE dno = '10';  
← ORA-02291: 무결성 제약 조건.... 에러 발생
```

```
SQL> UPDATE emp SET dno = '20' WHERE ename = '문시현';  
← ORA-02291: 무결성 제약 조건.... 에러 발생
```

- 제약 조건은 입력(수정, 삭제)작업시에 발생한다.
- 오라클에서 제약 조건은 지연 가능하다.

테이블과 ERD (Entity Relationship Diagram)

- dept, emp 테이블의 ERD



테이블 상세 도표

dept 테이블

컬럼명	dno	dname	loc
PK/UK/NOT NULL	PK		
참조 테이블			
참조 컬럼			
CHECK			
데이터 타입	VARCHAR2	VARCHAR2	VARCHAR2
길이	2	15	9

emp 테이블

컬럼명	eno	ename	sex	job	mgr	hdate	sal	comm	dno
PK/UK/NOT NULL	PK								
참조 테이블					emp				dept
참조 컬럼					eno				dno
CHECK									
데이터 타입	VARCHAR2	VARCHAR2	VARCHAR2	VARCHAR2	VARCHAR2	DATE	NUMBER	NUMBER	VARCHAR2
길이	4	15	4	12	4				2

실습

1. 다음 구조를 갖는 테이블을 생성한다. 각 테이블에는 필요한 PK와 FK를 지정한다
 - 데이터 타입이나 길이는 임의로 각자 결정하고 테이블 생성 순서에 주의한다.

factory 테이블(공장)

: fno(공장번호), fname(공장이름), loc(지역)

goods 테이블(제품)

: gno(제품번호), gname(제품명), pri(표준단가), fac_no(생산공장)

prod 테이블(출고 상품)

: s_num(일련번호), gno(제품번호), pri (출고단가), pdate(생산일자)

제약 조건 2.
UK, NOT NULL, CHECK

학습목표

1. UK, CHECK, NOT NULL 제약 조건 설정을 학습한다.
2. 각 제약 조건에 의한 무결성을 확인한다.

UK(Unique Key), NOT NULL 설정

```
CREATE TABLE 테이블 (  
.....  
CONSTRAINT 제약_조건 UNIQUE (컬럼));  
  
CREATE TABLE 테이블 (  
컬럼 데이터_타입 CONSTRAINT 제약_조건 NOT NULL,  
.....);
```

```
CREATE TABLE 테이블 (  
컬럼 데이터_타입 CONSTRAINT 제약_조건 UNIQUE,  
.....);
```

- UK는 고유 인덱스가 만들어진다.
- NOT NULL 컬럼 레벨에서만 정의할 수 있다.

CHECK 설정

```
CREATE TABLE 테이블 (  
.....  
CONSTRAINT 제약_조건 CHECK (조건));
```

```
CREATE TABLE 테이블 (  
  컬럼 데이터_타입 CONSTRAINT 제약_조건 CHECK (조건),  
  ...;
```

- 행에 입력될 데이터의 조건을 정의한다.
- 조건은 WHERE절에 기술하는 조건 형식과 동일하다.
- NOT NULL은 CHECK 제약 조건으로 간주된다.
 - . CHECK (컬럼 IS NOT NULL)

- CHECK와 NOT NULL 제약 조건 내용 검색

```
SELECT constraint_name, search_condition FROM user_constraints  
WHERE table_name = '테이블';
```

- che.sql로 저장한다.

UK, CHECK, NOT NULL을 적용한 테이블 생성

- 다음 상세 도표에 따라 테이블을 생성한다.

emp3 테이블

컬럼명	eno (사번)	ename (이름)	gno (주민번호)	sex (성별)
PK/UK/NOT NULL	PK	NOT NULL	UK	
참조 테이블				
참조 컬럼				
CHECK			LENGTH(13)	(여, 남)
데이터 타입	VARCHAR2	VARCHAR2	VARCHAR2	VARCHAR2
길이	4	50	13	4

UK, CHECK, NOT NULL을 적용한 테이블 생성

```
SQL> CREATE TABLE emp3 (  
2   eno VARCHAR2(4),  
3   ename VARCHAR2(50) CONSTRAINT emp3_ename_nu NOT NULL,  
4   gno VARCHAR2(13),  
5   sex VARCHAR2(2),  
6   CONSTRAINT emp3_eno_pk PRIMARY KEY (eno),  
7   CONSTRAINT emp3_gno_uk UNIQUE (gno),  
8   CONSTRAINT emp3_gno_ch CHECK (LENGTH(gno)=13),  
9   CONSTRAINT emp3_sex_ch CHECK (sex IN ('여','남')));
```

- 생성 후 제약 조건을 확인한다.

```
SQL> @ucon  
SQL> @ref
```

추가된 제약 조건 확인

```
SQL> INSERT INTO emp3 (eno, ename, gno, sex)
2 VALUES ('1001', '문시현', '0602183123456', '남');
```

```
SQL> INSERT INTO emp3 (eno, ename, gno, sex)
2 VALUES ('1002', NULL, '0605123123456', '남');
```

← ORA-01400 에러, ename null

```
SQL> INSERT INTO emp3 (eno, ename, gno, sex)
2 VALUES ('1003', '김민채', '0602183123456', '여');
```

← ORA-00001 에러, gno 중복

```
SQL> INSERT INTO emp3 (eno, ename, gno, sex)
2 VALUES ('1004', '권아현', '060121312345', '여');
```

← ORA-02290 에러, gno 길이

```
SQL> INSERT INTO emp3 (eno, ename, gno, sex)
2 VALUES ('1005', '권석현', '0601213123456', 'M');
```

← ORA-02290 에러, sex 입력

실습

1. 아래 도표를 보고 테이블을 생성한다.
 - 테이블 명에 한글을 허용한다.

제품

컬럼명	제품번호	제품명	제품단가
PK/UK/NOT NULL	PK	UK	
참조 테이블			
참조 컬럼			
CHECK			> 0
데이터 타입	VARCHAR2	VARCHAR2	NUMBER
길이	12	100	

판매전표

컬럼명	전표번호	판매일자	고객명	총액
PK/UK/NOT NULL	PK	NOT NULL	NOT NULL	
참조 테이블				
참조 컬럼				
CHECK				> 0
데이터 타입	VARCHAR2	DATE	VARCHAR2	NUMBER
길이	12		50	

실습

전표상세

컬럼명	전표번호	제품번호	수량	단가	금액
PK/UK/NOT NULL	PK		NOT NULL	NOT NULL	NOT NULL
참조 테이블	판매전표	제품			
참조 컬럼	전표번호	제품번호			
CHECK					> 0
데이터 타입	VARCHAR2	VARCHAR2	NUMBER	NUMBER	NUMBER
길이	12	12			

제약 조건 관리 추가 삭제와 비활성화

학습목표

1. 제약 조건을 추가하고 삭제하는 과정을 학습한다.
2. 제약 조건 비활성화/활성화 과정과 이로 인한 변화를 학습한다.
3. 제약 조건 삭제나 비활성화시에 인덱스 변화를 확인한다.

제약 조건 추가 삭제

```
ALTER TABLE 테이블  
ADD CONSTRAINT 제약_조건 제약_조건_타입;
```

```
ALTER TABLE 테이블  
MODIFY 컬럼 CONSTRAINT 제약_조건 NOT NULL;
```

```
ALTER TABLE 테이블  
DROP PRIMARY KEY | UNIQUE(컬럼) | CONSTRAINT 제약_조건 [CASCADE];
```

- NOT NULL을 제외하고 모두 ADD 명령으로 제약 조건을 추가한다.
- NOT NULL은 MODIFY 명령으로 추가한다.
- PK나 UK는 삭제시에 참조 중인 FK를 먼저 삭제해야 한다.
 - . CASCADE를 이용할 수 있다.
- 제약 조건은 'CREATE TABLE' 명령이 아니라 'ALTER TABLE ~' 통해서 설정하는 것이 작업에 유리하다.

제약 조건 활성화 비활성화

```
ALTER TABLE 테이블  
ENABLE CONSTRAINT 제약_조건;
```

```
ALTER TABLE 테이블  
DISABLE CONSTRAINT 제약_조건 [CASCADE]
```

- PK와 UK가 ENABLE/DISABLE 될 때 인덱스도 생성/삭제된다.
- PK와 UK가 DISABLE 되려면 참조하는 FK가 지워지거나 비활성화 되어야 한다.
 - . CASCADE 옵션을 이용 할 수 있다.

제약 조건 추가

- 다음 도표를 보고 테이블을 생성한다.
 - 제약 조건은 ALTER TABLE 명령으로 추가한다.

class(강좌)

컬럼명	cno (강좌번호)	cname (강좌명)
PK/UK/NOT NULL	PK	UK/ NOT NULL
참조 테이블		
참조 컬럼		
CHECK		
데이터 타입	VARCHAR2	VARCHAR2
길이	4	50

st(학생)

컬럼명	sno (학생번호)	sname (이름)	cno (강좌번호)
PK/UK/NOT NULL	PK		
참조 테이블			class
참조 컬럼			cno
CHECK			
데이터 타입	VARCHAR2	VARCHAR2	VARCHAR2
길이	4	50	4

제약 조건 추가

```
SQL> CREATE TABLE class (  
2   cno VARCHAR2(4),  
3   cname VARCHAR2(50));
```

```
SQL> CREATE TABLE st (  
2   sno VARCHAR2(4),  
3   sname VARCHAR2(50),  
4   cno VARCHAR2(4));
```

```
SQL> ALTER TABLE class  
2   ADD CONSTRAINT class_cno_pk PRIMARY KEY (cno);
```

```
SQL> ALTER TABLE class  
2   ADD CONSTRAINT class_cname_uk UNIQUE (cname);
```

제약 조건 추가

```
SQL> ALTER TABLE class  
2  MODIFY cname CONSTRAINT class_cname_nu NOT NULL;
```

```
SQL> ALTER TABLE st  
2  ADD CONSTRAINT st_sno_pk PRIMARY KEY (sno);
```

```
SQL> ALTER TABLE st  
2  ADD CONSTRAINT st_cno_fk FOREIGN KEY (cno) REFERENCES class (cno);
```

- 제약 조건 추가 후에 반드시 생성을 확인한다.

제약 조건 삭제

- class와 st에 추가된 제약조건을 모두 삭제한다.

```
SQL> ALTER TABLE st  
2 DROP PRIMARY KEY;
```

```
SQL> ALTER TABLE st  
2 DROP CONSTRAINT st_cno_fk;
```

```
SQL> ALTER TABLE class  
2 DROP PRIMARY KEY;
```

```
SQL> ALTER TABLE class  
2 DROP CONSTRAINT class_cname_uk;
```

```
SQL> ALTER TABLE class  
2 DROP CONSTRAINT class_cname_nu;
```

- 삭제 순서에 주의한다.
- 다음 실습을 위해 제약 조건을 다시 추가한다.

제약 조건 비활성화

- 제약 조건의 상태를 확인하고 비활성화/활성화 한다.

```
SQL> SELECT table_name, constraint_name, status  
2 FROM user_constraints  
3 WHERE table_name IN ('CLASS','ST');
```

```
SQL> ALTER TABLE st  
2 DISABLE CONSTRAINT st_sno_pk;
```

```
SQL> INSERT INTO st (sno, sname) values ('10','조조');  
SQL> INSERT INTO st (sno, sname) values ('10','유비');
```

```
SQL> SELECT * FROM st;
```

실습

1. 아래 도표를 보고 테이블을 생성한다.
 - 테이블 명에 한글을 허용한다.

제품

컬럼명	제품번호	제품명	제품단가
PK/UK/NOT NULL	PK	UK	
참조 테이블			
참조 컬럼			
CHECK			> 0
데이터 타입	VARCHAR2	VARCHAR2	NUMBER
길이	12	100	

판매전표

컬럼명	전표번호	판매일자	고객명	총액
PK/UK/NOT NULL	PK	NOT NULL	NOT NULL	
참조 테이블				
참조 컬럼				
CHECK				> 0
데이터 타입	VARCHAR2	DATE	VARCHAR2	NUMBER
길이	12		50	

실습

전표상세

컬럼명	전표번호	제품번호	수량	단가	금액
PK/UK/NOT NULL	PK		NOT NULL	NOT NULL	NOT NULL
참조 테이블	판매전표	제품			
참조 컬럼	전표번호	제품번호			
CHECK					> 0
데이터 타입	VARCHAR2	VARCHAR2	NUMBER	NUMBER	NUMBER
길이	12	12			

인덱스(Index) 생성

인덱스 구조와 이해

학습목표

1. 인덱스 구조에 대해서 학습한다.
2. SQL문과 인덱스 사용에 대해 학습한다.
3. 인덱스 생성과 삭제에 대해 학습한다.

인덱스

```
SELECT ...  
FROM 학생  
WHERE 이름 = '사마의';
```

```
SELECT ...  
FROM 학생  
WHERE 학번 = '003';
```

학번 인덱스

학번	ROWID
001	AABBA
002	AABBB
003	AABBC
004	AABBD
005	AABBE
006	AABCA
....	

학생

학번	이름	
201	조조	
005	유비	
114	장비	
321	관우	
003	사마의	
101	조운	
...	...	

인덱스

- 인덱스 종류
 - 고유
 - 고유 인덱스(Unique Index), 비 고유 인덱스(Non unique Index)
 - 물리 구조
 - B-Tree Index, bitmap Index
- 인덱스 생성
 - 자동 생성
 - . PK, UK로 지정된 컬럼은 자동으로 생성 된다.
 - . Unique Index가 생성된다.
 - 수동 생성
 - . CREATE INDEX 명령을 통해 직접 생성한다.
 - . Non unique Index 가 생성된다.
 - . FK 컬럼에 반드시 생성한다.

Index 생성과 삭제

```
CREATE INDEX 인덱스  
ON 테이블 (컬럼 | 함수 | 수식);
```

```
DROP INDEX 인덱스;
```

- 인덱스 생성 조건
 - 전체 데이터의 10%~15% 정도의 데이터를 검색하는 경우
 - 검색 데이터가 한 번에 확인이 가능한 정도라면 의미가 있다.
 - WHERE 절이나 조인에 사용되는 컬럼
 - 데이터의 행이 매우 많은 경우

인덱스 확인

```
SELECT c.index_name, c.column_name, c.column_position,  
       i.uniqueness  
FROM user_indexes i, user_ind_columns c  
WHERE c.index_name = i.index_name;  
  
SELECT index_name, column_expression  
FROM user_ind_expressions;
```

- Column_position : 여러 컬럼으로 인덱스가 생성된 경우 컬럼의 순서
- Column_expressions : 수식에 의해 만들어진 인덱스의 해당 수식

인덱스 확인

- 제약조건을 추가하고 인덱스를 확인해 보자.
 - school_con.sql 파일을 실행하고 인덱스를 확인한다.

```
SQL> SELECT i.table_name, i.index_name, c.column_name,  
2         c.column_position, i.uniqueness  
3 FROM user_indexes i, user_ind_columns c  
4 WHERE c.index_name = i.index_name  
5 AND i.table_name IN ('STUDENT', 'PROFESSOR', 'COURSE', 'SCORE')  
6 ORDER BY i.table_name;
```

- PK(UK)에 의해서 자동으로 생성된 인덱스의 이름은 PK(UK)와 동일하다.
- SCORE 테이블의 PK는 SNO, CNO컬럼으로 구성되어 컬럼의 순서가 매우 중요하다.

인덱스 생성 - FK

- FK 컬럼에 인덱스를 추가한다.

```
SQL> CREATE INDEX course_pno_fk  
2  ON course (pno);
```

```
SQL> CREATE INDEX score_cno_fk  
2  ON score (cno);
```

```
SQL> SELECT i.table_name, i.index_name, c.column_name  
2  FROM user_indexes i, user_ind_columns c  
3  WHERE c.index_name = i.index_name  
4  AND i.index_name LIKE '%FK'  
5  ORDER BY c.table_name;
```

- FK는 인덱스가 자동으로 만들어지지 않는다.
- SCORE의 SNO는 기존 PK 인덱스가 있으므로 생성하지 않는다.

인덱스 생성

- STUDENT 테이블에 다양한 인덱스를 생성하고 조회한다.

```
SQL> CREATE INDEX student_sname_idx  
2 ON student (sname);
```

```
SQL> CREATE INDEX student_major_sname_idx  
2 ON student (major, sname);
```

```
SQL> CREATE INDEX student_coavr_idx  
2 ON student (avr*4.5/4.0);
```

```
SQL> CREATE INDEX pr_ord_idx  
2 ON professor (SUBSTR(orders,1,1));
```

- 여러 컬럼으로 인덱스가 만들어 질 때 구조는 해당 컬럼으로 정렬하는 경우와 동일하다.
- 인덱스는 컬럼 뿐 아니라 수식으로도 생성 가능하다.

인덱스 확인

- STUDENT 테이블의 인덱스를 확인한다.

```
SQL> SELECT c.index_name, c.column_name, c.column_position  
2   FROM user_indexes i, user_ind_columns c  
3   WHERE c.index_name = i.index_name  
4   AND c.table_name = 'STUDENT'  
5   ORDER BY c.index_name, c.column_position;
```

```
SQL> SELECT index_name, column_expression  
2   FROM user_ind_expressions  
3   WHERE index_name = 'STUDENT_COAVR_INDX';
```

- 수식을 이용한 인덱스의 경우 COLUMN_NAME의 값이 일련번호로 검색된다.
- 수식은 USER_IND_EXPRESSIONS에서 검색한다.

인덱스 삭제

```
SQL> DROP INDEX student_major_sname_indx;
```

```
SQL> DROP INDEX student_sno_pk;
```

← ORA-02429 에러 발생

- PK나 UK에 의해 생성된 인덱스는 직접 삭제 할 수 없다.

```
SQL> ALTER TABLE student  
2  DISABLE CONSTRAINT student_sno_pk CASCADE;
```

- PK나 UK를 삭제하거나 비활성화 하면 인덱스가 삭제된다.

실습

1. 이전 강의 실습에서 생성한 테이블에 필요한 인덱스를 추가한다.

제품

컬럼명	제품번호	제품명	제품단가
PK/UK/NOT NULL	PK	UK	
참조 테이블			
참조 컬럼			
CHECK			> 0
데이터 타입	VARCHAR2	VARCHAR2	VARCHAR2
길이	12	100	

판매전표

컬럼명	전표번호	판매일자	고객명	총액
PK/UK/NOT NULL	PK	NOT NULL	NOT NULL	
참조 테이블				
참조 컬럼				
CHECK				> 0
데이터 타입	VARCHAR2	DATE	VARCHAR2	VARCHAR2
길이	12		50	

실습

전표상세

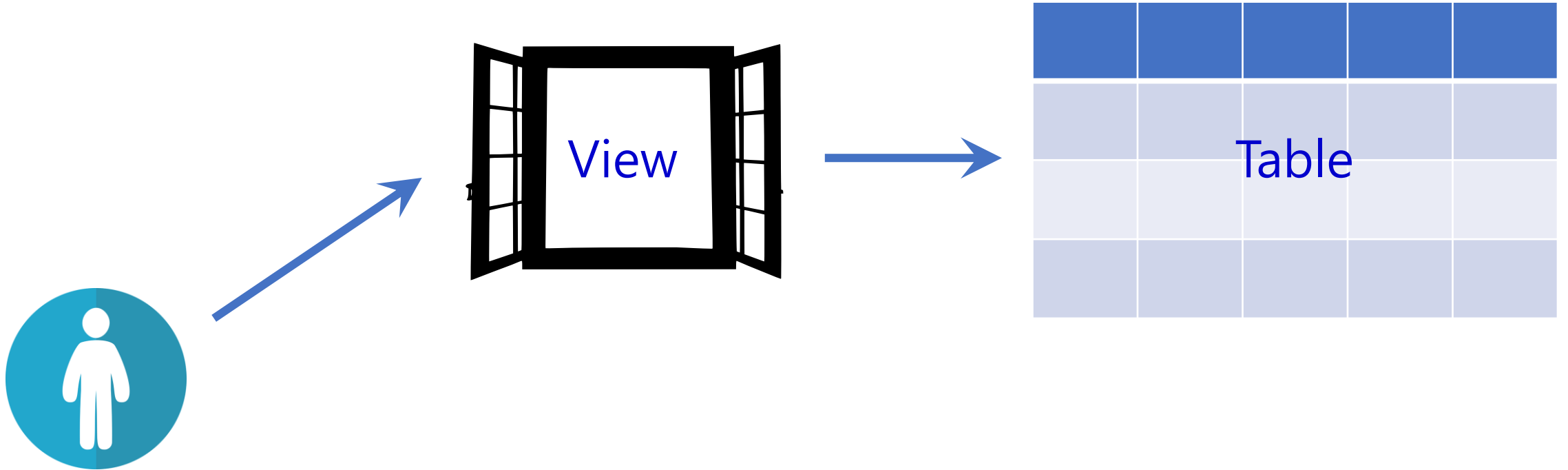
컬럼명	전표번호	제품번호	수량	단가	금액
PK/UK/NOT NULL	PK		NOT NULL	NOT NULL	NOT NULL
참조 테이블	판매전표	제품			
참조 컬럼	전표번호	제품번호			
CHECK					> 0
데이터 타입	VARCHAR2	VARCHAR2	NUMBER	NUMBER	NUMBER
길이	12	12			

뷰 (View)

학습목표

1. 뷰에 대해서 학습한다.
2. 뷰 생성 옵션에 대해 학습한다.
3. 인라인 뷰(Inline View)에 대해 학습한다.

뷰(View)



- 뷰의 종류
 - 단순 뷰(Simple view)
 - . DML문장 수행 가능
 - 복합 뷰(Complex view)
 - . 다중 테이블이나 수식 등이 포함된 뷰

뷰 생성

```
CREATE [OR REPLACE][FORCE | NOFORCE] VIEW 뷰 (컬럼 ... )  
AS (SELECT 문장 : sub query)  
[WITH CHECK OPTION [CONSTRAINT constraint_name]]  
[WITH READ ONLY [CONSTRAINT constraint_name]];
```

- OR REPLACE
 - : ALTER VIEW를 대체한다.
 - : SEGMENT가 아님
- FORCE
 - : 기반 테이블 없이 뷰를 생성한다.
- WITH CHECK OPTION
 - : 뷰에 의해서 검색 가능한 행만 DML 작업이 가능하도록 제약한다.
- WITH READ ONLY
 - : 뷰를 통해서만 SELECT만 가능하도록 제한한다.

뷰 삭제와 확인

```
DROP VIEW 뷰;
```

```
SELECT view_name, text FROM user_views;
```

- text
 - : 뷰 생성 서브 쿼리
 - : LONG 타입으로 설정이 필요하다.

- 뷰 생성 주의 사항

- 뷰를 생성할 수 있는 'CREATE VIEW' 권한이 보통은 RESOURCE, CONNECT 롤에 포함되어 있지 않음으로 사용자에게 반드시 해당 권한을 할당해야 한다.
- 사용자에게 직접 할당하거나 할당된 RESOURCE, CONNECT 롤에 할당한다.
- 오라클 19C를 설치하고 환경을 구성하는 첫 번째 강좌에서 수행되어 있다.
 - . 3. 실습 환경 설정 – 권한 할당
 - . GRANT create view TO resource;

뷰 생성

- 일반화학 과목의 학과별 기말고사 평균을 뷰로 생성한다.

```
SQL> CREATE VIEW ma_result (과목번호, 과목명, 학과, 기말고사평균)
2  AS SELECT c.cno, cname, major, ROUND(AVG(result))
3      FROM student s, course c, score r
4      WHERE s.sno=r.sno AND r.cno=c.cno
5      AND cname='일반화학'
6      GROUP BY c.cno, cname, major;
```

```
SQL> SET LONG 1000      ← 1000 byte까지 출력
```

```
SQL> SELECT view_name, text FROM user_views;
```

```
SQL> DESC ma_result;
```

```
SQL> SELECT * FROM ma_result;
```

WITH CHECK 옵션

```
SQL> CREATE VIEW st_ch  
2 AS SELECT sno, sname, syear, avr  
3 FROM student  
4 WHERE syear = 1;
```

```
SQL> INSERT INTO st_ch VALUES ('000001', '시현', 2, 4.0);
```

```
SQL> SELECT * FROM student WHERE sname = '시현';
```

```
SQL> SELECT * FROM st_ch WHERE sname = '시현'; ← 검색되지 않는다.
```

- 뷰에 보이지 않는 행이 뷰를 통해 입력될 수 있다.

WITH CHECK 옵션

```
SQL> CREATE OR REPLACE VIEW st_ch  
2 AS SELECT sno, sname, syear, avr  
3 FROM student  
4 WHERE syear = 1  
5 WITH CHECK OPTION CONSTRAINT view_st_ch_ck;
```

```
SQL> INSERT INTO st_ch VALUES ('000001', '시현', 2, 4.0);  
INSERT INTO st_ch VALUES ('000001', '시현', 2, 4.0)  
*
```

1행에 오류:

ORA-01402: 뷰의 WITH CHECK OPTION의 조건에 위배 됩니다

인라인 뷰(Inline View)

```
SELECT ...  
FROM (SELECT 문장 : sub query) [별명]
```

- 인라인 뷰는 FROM절에 쓰인 서브쿼리다.
- 인라인 뷰는 문장이 실행되는 동안만 존재하는 뷰이다.
- 인라인 뷰는 일반 뷰와 동일하게 조인이나 검색에 이용된다.

인라인 뷰 사용

- 각 부서별 최소 급여자를 검색한다.

```
SQL> SELECT eno, ename, d.dno, sal, msal  
2  FROM emp e,  
3      (SELECT dno, MIN(sal) msal FROM emp  
4      GROUP BY dno) d  
5  WHERE e.dno = d.dno  
6  AND sal = msal;
```

실습

1. 사원의 년봉을 검색할 수 있는 뷰를 생성한다.
2. 학생의 학점 4.5 만점으로 환산된 정보를 검색할 수 있는 뷰를 생성한다.
3. 각 과목별 평균 점수를 검색할 수 있는 뷰를 생성한다.
4. 각 직원과 관리자의 이름을 검색할 수 있는 뷰를 생성한다.
5. 각 과목별로 평가 점수가 F인 학생을 검색할 뷰를 생성한다.
6. 각 과목별로 기말고사가 일등인 학생의 명단을 검색할 뷰를 생성한다.
7. 물리학과 교수의 과목을 수강하는 학생의 명단을 검색할 뷰를 생성한다.

Top-N 분석과 RANK

학습목표

1. 인라인 뷰를 이용한 Top-N 분석을 학습한다.
2. RANK()를 이용해서 순위를 검색한다.

TOP-N 분석

- 정렬된 결과의 일부를 검색하는 방법은 다양하지만 가장 기본적인 방법은 인라인 뷰를 이용하는 것이다.
- 인라인 뷰를 이용하는 이유는 SQL문에서 정렬이 가장 마지막에 수행되기 때문이다.

```
SELECT  ROWNUM, [컬럼, 컬럼, ...]  
FROM    (SELECT  ...  ORDER BY ...)  
WHERE   ROWNUM <= N;
```

- 상/하위 N개의 행을 검색한다.
- 인라인 뷰에 반드시 ORDER BY절이 필요하다.
- ROWNUM은 검색된 행의 순서이다.

인라인 뷰를 이용한 Top-N 분석

- 가장 높은 급여를 받는 3인을 검색한다.

```
SQL> SELECT ROWNUM, eno, ename, sal  
2   FROM (SELECT eno, ename, sal  
3           FROM emp  
4           ORDER BY sal DESC)  
5  WHERE ROWNUM <= 6;
```

- 다음 SELECT문과 비교해 본다.

```
SQL> SELECT ROWNUM, eno, ename, sal  
2   FROM emp  
3  WHERE ROWNUM <= 6  
4  ORDER BY sal DESC;
```

일반 뷰 활용

- 정렬된 뷰를 생성하고 ROWNUM을 이용 검색한다.

```
SQL> CREATE VIEW semp  
2 AS SELECT eno, ename, sal FROM emp  
3 ORDER BY sal DESC;
```

```
SQL> SELECT * FROM semp  
2 WHERE ROWNUM <= 5;
```

- 해당 업무의 지속성에 따라 인라인 뷰 또는 뷰를 이용한다.

RANK() 를 이용한 순위 검색

```
RANK()          OVER ([PARTITION BY 컬럼, 컬럼 ...] ORDER BY .....)  
DENSE_RANK()    OVER ([PARTITION BY 컬럼, 컬럼 ...] ORDER BY .....)  
ROW_NUMBER()    OVER ([PARTITION BY 컬럼, 컬럼 ...] ORDER BY .....)
```

- RANK 함수를 이용해서 다양한 순위를 검색한다.
- RANK()
 - . 순위 검색에서 중복 순위만큼 값을 증가시킨다.
- DENSE_RANK()
 - . 동일 순위를 하나의 순위 및 행으로 취급한다.
- ROW_NUMBER()
 - . 동일 값이라도 반드시 순위를 다르게 결정한다.
 - . 정렬 값이 동일한 경우 어떤 값이 우선이 될 지 알 수 없다.
 - . 오라클의 경우 ROWID에 따라 결정된다.
 - . 동일한 값이 없도록 정렬을 추가하는 것이 좋다.

Ranking 검색

- 급여 순위를 검색한다.

```
SQL> SELECT eno, ename, sal,  
2          RANK() OVER (ORDER BY sal DESC) RANK,  
3          DENSE_RANK() OVER (ORDER BY sal DESC) DRANK,  
4          ROW_NUMBER() OVER (ORDER BY sal DESC) RRANK  
5 FROM emp;
```

- 부서별 급여 순위를 검색한다.

```
SQL> SELECT eno, ename, dno, sal,  
2          RANK() OVER (PARTITION BY dno ORDER BY sal DESC) RANK  
3 FROM emp;
```

Ranking 검색

- 사원의 년봉을 순위별로 검색한다.

```
SQL> SELECT RANK() OVER (ORDER BY sal*12+NVL(comm,0) DESC) RRANK,  
2          eno, ename, sal*12+NVL(comm,0) 연봉  
3 FROM emp;
```

- 부서별로 사원의 급여 순위를 검색한다.
 - 부서명, 급여를 반드시 검색한다.

```
SQL> SELECT RANK() OVER (PARTITION BY d.dno ORDER BY sal DESC) RRANK,  
2          d.dno, dname, eno, ename, sal  
3 FROM dept d, emp e  
4 WHERE d.dno=e.dno;
```


실습

1. 4.5 환산 평점이 가장 높은 3인의 학생을 검색한다.
2. 화학과 학생 중 기말고사 유기화학 점수가 최하위인 학생 3명을 검색한다.
3. 기말고사 과목별 평균이 높은 3과목을 검색한다.
4. 학과별 학년별 기말고사 평균이 순위 3위까지를 검색한다.
 - (학과, 학년, 평균점수를 검색한다.)
5. 기말고사 성적이 높은 과목을 담당하는 교수 3인을 검색한다.
 - (교수 이름, 과목명과 평균점수를 검색한다.)
6. 학과별 기말고사 평균의 순위를 검색한다.
7. 화학과 학생의 각 과목별 기말고사 점수 순위를 검색한다.

시퀀스(Sequence)

학습목표

1. 시퀀스에 대해 학습한다.
2. 시퀀스의 용도에 대해 학습한다.
3. 시퀀스를 생성하고 값을 추출한다.

시퀀스(Sequence)

- 유일한 값을 자동으로 생성한다.
- 보통 PK 값을 생성하는데 사용된다.
- 응용 프로그램을 대체한다.
- 캐시를 통해 성능을 높인다.

시퀀스 생성과 수정

```
CREATE [| ALTER] SEQUENCE 시퀀스  
[ START WITH 시작_값  
  INCREMENT BY 증가_값  
  MAXVALUE [상한_값 | NOMAXVALUE]  
  MINVALUE [하한_값 | NOMINVALUE]  
  CYCLE | NOCYCLE  
  CACHE [cache_개수| NOCACHE] ];
```

- START WITH : 할당을 시작하는 값을 정의한다.
- INCREMENT BY : 생성되는 값의 증가치를 결정한다. 지정하지 않으면 1에서 시작한다.
- MAXVALUE : 생성될 상한 값을 정의한다. 기본 설정은 NOMAXVALUE이다.
- MINVALUE : 하한값을 정의한다. 기본 설정은 NOMINVALUE이다.
- CYCLE : 상한 값까지 생성한 이후 다시 하한 값부터 재 생성할 것인지 여부이다.
- CACHE : 메모리에 미리 생성될 값의 개수를 정의한다.

시퀀스 삭제와 확인, 사용

```
DROP SEQUENCE 시퀀스;
```

```
SELECT sequence_name, max_value, min_value, increment_by,  
       cache_size, last_number, cycle_flag  
FROM user_sequences;
```

```
.....시퀀스.NEXTVAL .....
```

```
.....시퀀스.CURRVAL .....
```

- NEXTVAL : 시퀀스로부터 유일한 값을 할당 받는다.
- CURRVAL : 시퀀스로부터 마지막으로 할당 받은 값을 다시 할당 받는다.
- 주로 INSERT문의 VALUES절이나 UPDATE의 SET절에서 많이 사용된다.

시퀀스 생성

- 시퀀스를 생성하고 설정을 검색한다.

```
SQL> CREATE SEQUENCE emp_eno_seq  
2  START WITH 1  
3  INCREMENT BY 1  
4  NOMAXVALUE  
5  NOMINVALUE  
6  NOCYCLE  
7  CACHE 20;
```

```
SQL> CREATE SEQUENCE dept_dno_seq;
```

```
SQL> SELECT sequence_name, max_value, min_value, increment_by,  
2          cache_size, last_number, cycle_flag  
3  FROM user_sequences;
```

시퀀스 이용

- emp table에 시퀀스를 이용해서 행을 입력한다.

```
SQL> INSERT INTO emp (eno, ename)
  2  VALUES (emp_eno_seq.nextval, '첫 번째');
```

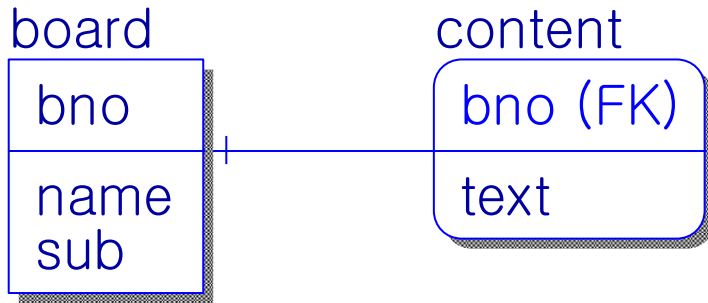
```
SQL> SELECT emp_eno_seq.currval FROM dual;
```

```
SQL> ROLLBACK;
```

```
SQL> SELECT sequence_name, max_value, min_value, increment_by,
  2          cache_size, last_number, cycle_flag
  3  FROM user_sequences;
```


시퀀스 이용

- 다음 ERD에 따라 생성된 테이블에 행을 입력한다.
 - 테이블과 제약조건 생성 SQL은 board.sql 파일에 제공된다.



```
SQL> CREATE SEQUENCE bo_bno_seq;
```

```
SQL> INSERT INTO board (bno, name, sub)  
2  VALUES (bo_bno_seq.nextval, '이름', '제목은');
```

```
SQL> INSERT INTO content (bno, text)  
2  VALUES (bo_bno_seq.currval, '본문');
```